

Critical Thinking Questions

#1 |

What is a GUI?

A GUI is a Graphical User Interface(sometimes pronounced gooey). Not only does a GUI use components such as frames, buttons, and text fields to communicate with the user, but GUIs are event-driven. An event-driven application executes code in response to events. A GUI responds to an event by executing a method called an event handler. For example, when the user clicks a button, the application must be able to determine which button was clicked and then execute the code that relates to that button.

#2 |

Explain how code is executed in an event-driven application.

In an event-driven Java application, code doesn't run in a straight, top-to-bottom sequence like a simple main() program. Instead, execution is driven by events (user actions, system changes, messages, etc.) and handled by listeners or callbacks.

#3 |

Can components be added directly to a frame? Explain.

Yes, you can put components directly into a JFrame. to add components to a JFrame , you add them to the frame's content pane. Two common techniques are used to provide the components for a frame's content pane: Create a container such as a JPanel , a JScrollPane , or a JTabbedPane , add components to it, then use JFrame.

#4 |

Can a label respond to events? Explain.

Yes, a label can respond to certain events in Java GUI frameworks like Swing, usually used for mouse events.

#5 |

Why do you think a GUI needs to be run from an event-dispatching thread?

Swing event handling code runs on a special thread known as the event dispatch thread. Most code that invokes Swing methods also runs on this thread. This is necessary because most Swing object methods are not "thread safe": invoking them from multiple threads risks thread interference or memory consistency errors. Some Swing component methods are labelled "thread safe" in the API specification; these can be safely invoked from any thread. All other Swing component methods must be invoked from the event dispatch thread. Programs that ignore this rule may function correctly most of the time, but are subject to unpredictable errors that are difficult to reproduce.

#6 |

What is the difference between a label and a button?

Labels are passive, non-interactive text elements used to identify or describe other interface components. Buttons are interactive elements that users click or tap to trigger actions, such as submitting a form or navigating,

#8 |

List three ways to control the layout of a content pane.

setLayout() method: `contentPane.setLayout(new BorderLayout())`

Null layout: Use `contentPane.setLayout(null)` for absolute positioning

Default BorderLayout: Rely on the JFrame content pane's default BorderLayout by adding components to regions like NORTH, SOUTH, CENTER.

#13 |

What must first be done with numeric data typed in a text field before it can be used in a calculation?

It has to be converted into a double value.

#14 |

What is the value of num1 in the last statement below?

```
Double num1;
```

```
Double num2 = new Double(3);
```

```
String num3 = "5";
```

```
Num1 = num2.doubleValue() + Double.valueOf(num3).doubleValue();
```

```
Num1 = 8.0
```

#15 |

An application prompts a user to select a name from a list of six names. Which is a better component choice: a text field or a combo-box?

A combo box is a better choice.

A combo box is designed for choosing one item from a small, fixed set of options and prevents invalid entries, which fits a list of six names much better than a free-form text field.

For example:

